

TinyDB Storage Engine

Goal: Build the physical, on-disk storage layer of a database from scratch: fixed-size pages with a slotted layout, a binary row codec, a buffer pool with LRU eviction, a write-ahead log with checksums, ARIES-style crash recovery (redo committed work, undo incomplete work), checkpoints, and page-level corruption detection — as a standalone project with real, working code for every piece.

1. What We Are Building

```
SQL / Execution (companion guide)
|
Transaction (Part 6)
|
Storage API (Part 9)
|
Page / Buffer / WAL (Parts 1-5)
|
FileChannel
|
Operating System / Disk
```

By the end of this guide you will have:

- A **fixed-size, slotted page** format with a real header, slot directory, and byte-level insert/read/delete logic.
- A **binary row codec** that encodes/decodes rows without relying on Java object serialization, including a null bitmap for compact null handling.
- A **buffer pool** with pin counts, dirty tracking, and real LRU eviction — no re-reading the same page from disk twice.
- A **write-ahead log** with LSNs, a real record format, checksums, and the enforced rule that a data page's changes are never flushed before the WAL record describing them is durable.
- **ARIES-style crash recovery**: scanning the WAL, redoing every committed change, undoing every incomplete transaction.
- **Checkpoints** that bound recovery time, and page checksums that turn silent corruption into a loud, immediate failure.
- A single **StorageEngine** facade that ties every layer together, ready to be handed to the companion Query Engine guide's executors and planner.

2. Learning Objectives

By the end of this guide you should be able to:

- Explain exactly why a slotted page layout lets row bytes move within a page without invalidating a **RowId** that points at them.
- Implement a binary row codec, including how a null bitmap avoids storing a full-width flag per column.
- Implement a real buffer pool with LRU eviction, including why a pinned page must never be evicted regardless of how stale its access time is.
- State the WAL-before-data-page rule precisely, and implement a **WalManager** that makes violating it structurally difficult.
- Implement ARIES-style crash recovery: scan, redo, undo — and explain why redo runs before undo, not after.
- Explain what a checkpoint actually bounds, and what page checksums can and cannot detect.

3. Why This Matters

"Design the physical storage layer of a database: how rows are laid out on disk, how you avoid re-reading the same page repeatedly, and how the system survives a crash without losing committed data or corrupting the file. Implement the crash recovery algorithm specifically."

This is one of the highest-signal database internals / infrastructure interview questions because durability claims are easy to assert and hard to actually implement correctly:

- **Byte-level thinking** — a slotted page, a null bitmap, and a WAL record format all require reasoning about exact byte offsets, not just object references.
- **The WAL-before-data-page rule** is a real, non-negotiable invariant, and violating it is the single most common way a "database" silently loses data on a crash while looking correct in every non-crash test.
- **Redo/undo recovery** is a genuinely subtle algorithm — many candidates can describe it in one sentence ("replay the log") without being able to say why redo must run before undo, or what "incomplete" means precisely.
- **Buffer pool design** connects directly to classic CS fundamentals (LRU, pinning) applied to a concrete, high-stakes resource-management problem.

4. Recommended Technology Stack

File I/O	java.nio.channels.FileChannel, java.nio.ByteBuffer
Checksums	java.util.zip.CRC32

5. Project Structure

```
tinydb-storage/
├── src/main/java/com/tinydb/storage/
│   ├── file/
│   │   ├── DatabaseDirectory.java
│   │   ├── TableFile.java
│   │   └── TableFileManager.java
│   ├── page/
│   │   ├── PageId.java, RowId.java
│   │   ├── PageHeader.java
│   │   ├── Slot.java
│   │   └── Page.java
│   ├── row/
│   │   ├── RowCodec.java
│   │   └── BinaryRowCodec.java
│   ├── buffer/
│   │   ├── BufferFrame.java
│   │   └── LruBufferPool.java
│   ├── fsm/
│   │   └── SimpleFreeSpaceMap.java
│   ├── table/
│   │   └── PageBasedTableHeap.java
│   ├── wal/
│   │   ├── Lsn.java, WalRecordType.java, WalRecord.java
│   │   └── FileWalManager.java
│   ├── recovery/
│   │   ├── Checkpoint.java
│   │   └── RecoveryManager.java
│   └── engine/
│       └── FileStorageEngine.java // the facade the SQL layer talks to
├── src/test/java/com/tinydb/storage/
│   ├── PageSlotDirectoryTest.java
│   ├── BinaryRowCodecTest.java
│   ├── LruBufferPoolTest.java
│   ├── PersistAcrossRestartTest.java
│   └── CrashRecoveryTest.java
```

6. High-Level Architecture: The Durability Chain

Every layer in this guide exists to uphold exactly one promise: once a write is acknowledged as committed, it survives a crash. The chain of responsibility that makes this true, end to end:

1. A row is encoded to bytes (Part 2) -- deterministic, no Java object serialization
2. The bytes are inserted into a page's slot directory (Part 1) -- in memory, via the buffer pool (Part 4)
3. BEFORE that page is ever flushed to disk, a WAL record describing the change is appended and made durable -- Part 5
4. On commit, the WAL's commit record is made durable (this is the actual moment the client may be told "success") -- Part 5
5. The data page itself is flushed to disk LATER, whenever convenient -- its own durability is not what the client waited on -- Part 4
6. If the process crashes at any point, recovery (Part 7) replays the WAL and reconstructs exactly the state that was promised -- Part 7

7. Phase 1 — The Page Size and Physical Addressing

A database file is divided into fixed-size pages, each addressed by a simple integer:

```
public final class StorageConstants {  
    public static final int PAGE_SIZE = 8192; // 8 KiB  
    private StorageConstants() { }  
}
```

physical byte offset of page N = $N \times \text{PAGE_SIZE}$

```
page 0 -> offset 0  
page 1 -> offset 8192  
page 2 -> offset 16384  
page 10 -> offset 81920
```

Every read or write this guide ever performs against a file ultimately reduces to this one multiplication — everything from here on is about what lives inside one 8192-byte page, and how to find the right page in the first place.

8. Phase 2 — PageId and RowId

```
public record PageId(String database, String table, long pageNumber) { }
```

A physical row is identified by which page it lives on and which slot within that page — not by its byte offset directly, which is free to change (§9 explains why):

```
public record RowId(long pageNumber, int slotNumber) { }
```

```
employees.tbl, page 12, slot 4 == new RowId(12, 4)
```

An index (companion Query Engine guide's §38) stores **indexedValue** → **RowId**, never a copy of the row itself — looking up a **RowId** and then reading that exact page/slot (§18) is the entire mechanism a point-lookup index relies on.

9. The Slotted Page Layout

```
+-----+
| PAGE HEADER (36 bytes, $10) |
+-----+
| SLOT 0  SLOT 1  SLOT 2  ... | <- slot directory, GROWS FORWARD
+-----+
|                               |
|          FREE SPACE          |
|                               |
+-----+
| ROW DATA  ROW DATA  ROW DATA | <- row bytes, GROWS BACKWARD from the page's end
+-----+
```

Each slot is a small, fixed-size (offset, length) pair pointing at where its row's actual bytes live, further down in the same page:

```
Slot 0 -> offset 8100, length 92
Slot 1 -> offset 7980, length 120
Slot 2 -> offset 7860, length 120
+-----+
```

This one level of indirection is the entire reason a slotted page exists: a row's bytes can be moved anywhere within the page (during a future compaction, for instance) by updating one slot entry, **without** changing the logical **Rowid** (§8) anything else in the system holds — indexes, in-flight scans, and cached row references all stay valid.

10. Phase 3 — The Page Header Format

Offset	Size	Field
0	8	page number
8	8	page LSN (S27 -- which WAL record last modified this page)
16	4	page type
20	4	slot count
24	4	free-space start (end of the slot directory)
28	4	free-space end (start of the row-data region)
32	4	checksum (S44)

```
+-----+
```

36 bytes total. This exact layout is the database's on-disk contract — it must be documented and versioned, because old database files must remain readable after the Java classes that read them evolve.

11. Phase 4 — The Page Class: Real Byte-Level Implementation

```
public final class Page {
    private static final int HEADER_SIZE = 36;
    private static final int SLOT_SIZE = 8; // 4 bytes offset + 4 bytes length
    static final int TOMBSTONE_LENGTH = -1; // marks a deleted slot -- see §32

    private final PageId pageld;
    private final ByteBuffer buffer;
    private boolean dirty;

    public Page(PageId pageld) {
        this.pageld = pageld;
        this.buffer = ByteBuffer.allocate(StorageConstants.PAGE_SIZE);
        initializeNewPage();
    }

    public Page(PageId pageld, ByteBuffer existingContent) {
        this.pageld = pageld;
        this.buffer = existingContent;
    }

    private void initializeNewPage() {
        setSlotCount(0);
        setFreeSpaceStart(HEADER_SIZE);
        setFreeSpaceEnd(StorageConstants.PAGE_SIZE);
        // -1 is a sentinel meaning "no LSN applied yet." WAL LSNs start at 0, so if we
        // initialized this to 0 instead, the redo idempotency check in the Recovery
        // Manager (page.pageLsn() >= record.lsn()) would wrongly treat LSN 0 as
        // "already applied" and skip the very first change ever made to this page.
        setPageLsn(-1);
    }

    public PageId pageld() { return pageld; }
    public ByteBuffer buffer() { return buffer; }
    public boolean dirty() { return dirty; }
    public void markDirty() { dirty = true; }
    public void clearDirty() { dirty = false; }

    // ---- Header field accessors ----

    public long pageLsn() { return buffer.getLong(8); }
    public void setPageLsn(long lsn) { buffer.putLong(8, lsn); markDirty(); }

    public int slotCount() { return buffer.getInt(20); }
    private void setSlotCount(int count) { buffer.putInt(20, count); }

    private int freeSpaceStart() { return buffer.getInt(24); }
    private void setFreeSpaceStart(int offset) { buffer.putInt(24, offset); }

    private int freeSpaceEnd() { return buffer.getInt(28); }
    private void setFreeSpaceEnd(int offset) { buffer.putInt(28, offset); }

    public int checksum() { return buffer.getInt(32); }
    public void setChecksum(int value) { buffer.putInt(32, value); } // does NOT markDirty() -- see §44
}
```

```

// ---- Slot directory ----
private int slotPosition(int slotNumber) { return HEADER_SIZE + slotNumber * SLOT_SIZE; }

private Slot readSlot(int slotNumber) {
    int pos = slotPosition(slotNumber);
    return new Slot(buffer.getInt(pos), buffer.getInt(pos + 4));
}

private void writeSlot(int slotNumber, int offset, int length) {
    int pos = slotPosition(slotNumber);
    buffer.putInt(pos, offset);
    buffer.putInt(pos + 4, length);
}

// ---- Row operations -- §17-§18 build the algorithms; here they're the real, tested implementation ----

public boolean hasSpaceFor(int rowLength) {
    return (freeSpaceEnd() - freeSpaceStart()) >= (rowLength + SLOT_SIZE);
}

public RowId insert(byte[] rowBytes) {
    if (!hasSpaceFor(rowBytes.length)) throw new PageFullException(pageId);

    int slotNumber = slotCount();
    int rowOffset = freeSpaceEnd() - rowBytes.length;

    buffer.put(rowOffset, rowBytes); // absolute put -- does not disturb the buffer's own position/limit
    writeSlot(slotNumber, rowOffset, rowBytes.length);

    setSlotCount(slotNumber + 1);
    setFreeSpaceStart(freeSpaceStart() + SLOT_SIZE);
    setFreeSpaceEnd(rowOffset);
    markDirty();

    return new RowId(pageId.pageNumber(), slotNumber);
}

/**
 * Redo-only variant of insert(): writes a row into a SPECIFIC, already-decided slot number
 * rather than allocating the next one, because the WAL record being replayed (§38) already
 * says exactly where this row landed the first time. Two cases: if this page's slot directory
 * doesn't yet have this slot (the page was never flushed after the original insert), it's
 * created exactly like insert() would, but pinned to the given slotNumber instead of the next
 * free one; if the slot already exists (a later, already-flushed change, or a prior redo pass
 * that got interrupted), this behaves exactly like restoreSlot() -- normal TinyDB operation
 * only ever assigns slot numbers sequentially, so no gap-filling case can arise in practice.
 */
public void insertAt(int slotNumber, byte[] rowBytes) {
    if (slotNumber < slotCount()) {
        restoreSlot(slotNumber, rowBytes); // slot already exists -- reuse the un-tombstone path
        return;
    }
    int rowOffset = freeSpaceEnd() - rowBytes.length;
    buffer.put(rowOffset, rowBytes);
    writeSlot(slotNumber, rowOffset, rowBytes.length);
    setSlotCount(slotNumber + 1);
    setFreeSpaceStart(freeSpaceStart() + SLOT_SIZE);
    setFreeSpaceEnd(rowOffset);
    markDirty();
}

```

```

public byte[] read(int slotNumber) {
    Slot slot = readSlot(slotNumber);
    if (slot.length() == TOMBSTONE_LENGTH) return null; // deleted -- §32
    byte[] rowBytes = new byte[slot.length()];
    buffer.get(slot.offset(), rowBytes); // absolute get
    return rowBytes;
}

public void update(int slotNumber, byte[] newRowBytes) {
    Slot slot = readSlot(slotNumber);
    if (newRowBytes.length != slot.length()) {
        // With fixed-size slots (every row in a table has the same TablePhysicalLayout.slotSize(),
        // companion Query Engine guide §34), this should never happen -- if it does, the caller
        // passed bytes for the wrong table's layout.
        throw new IllegalArgumentException("Row size mismatch: slot expects " + slot.length()
            + " bytes, got " + newRowBytes.length);
    }
    buffer.put(slot.offset(), newRowBytes);
    markDirty();
}

public void delete(int slotNumber) {
    Slot slot = readSlot(slotNumber);
    writeSlot(slotNumber, slot.offset(), TOMBSTONE_LENGTH); // tombstone -- see §32 for why not a
physical erase
    markDirty();
}

/**
 * Un-tombstones a slot by writing back a previously-captured before-image and restoring
 * its length. This is the one Page-level primitive that only the Recovery Manager's Undo
 * phase (§39) ever calls -- it exists solely to reverse a delete() during rollback of an
 * incomplete transaction. Undoing an INSERT reuses delete() (make the slot a tombstone
 * again) and undoing an UPDATE reuses update() (write the before-image back over the
 * after-image) because in both of those cases the slot was never tombstoned to begin with;
 * only undoing a DELETE needs to bring a tombstoned slot back to life, which is what makes
 * restoreSlot() necessary as its own method.
 */
public void restoreSlot(int slotNumber, byte[] beforeImage) {
    Slot slot = readSlot(slotNumber);
    buffer.put(slot.offset(), beforeImage);
    writeSlot(slotNumber, slot.offset(), beforeImage.length);
    markDirty();
}
}

```

```

public record Slot(int offset, int length) { }

```

Every method above operates through absolute **ByteBuffer** **get/put** calls (**buffer.put(rowOffset, rowBytes)**), not **buffer.position(rowOffset); buffer.put(rowBytes)** — deliberately, because the page's header fields and slot directory are read via their own absolute offsets throughout a row insert, and letting the buffer's internal position drift between those calls would be a correctness bug waiting to happen. **update** never needing to move a row within the page (§9's "why bytes can move" caveat notwithstanding) is a direct, deliberate consequence of every row in a table sharing one fixed slot size (companion guide §34) — there is no "row grew too big for its slot" case to handle here at all. Note that **restoreSlot** relies on that same fixed-slot-size guarantee: the **beforeImage** captured before a delete is always exactly **slot.length()** bytes, so writing it back can never overflow into the next row's data.

12. Phase 5 — Reading and Writing Pages via FileChannel

```
public final class PageIo {

    public Page readPage(FileChannel channel, PageId pageId) throws IOException {
        ByteBuffer buffer = ByteBuffer.allocate(StorageConstants.PAGE_SIZE);
        long offset = pageId.pageNumber() * StorageConstants.PAGE_SIZE;

        int totalRead = 0;
        while (buffer.hasRemaining()) {
            int read = channel.read(buffer, offset + totalRead);
            if (read < 0) break; // end of file
            totalRead += read;
        }
        if (totalRead == 0) throw new EOFException("Page does not exist: " + pageId);

        buffer.flip();
        Page page = new Page(pageId, buffer);
        verifyChecksum(page); // §44 -- fail loudly on corruption, never silently continue
        return page;
    }

    public void writePage(FileChannel channel, Page page) throws IOException {
        computeAndStoreChecksum(page); // §44 -- computed fresh on every write, right before it hits disk

        ByteBuffer buffer = page.buffer().duplicate();
        buffer.rewind();
        long offset = page.pageId().pageNumber() * StorageConstants.PAGE_SIZE;

        long position = offset;
        while (buffer.hasRemaining()) {
            int written = channel.write(buffer, position);
            if (written <= 0) throw new IOException("Unable to write page " + page.pageId());
            position += written;
        }
    }

    // verifyChecksum(...) / computeAndStoreChecksum(...) implemented in full in §44,
    // once CRC32 and the checksum field's exact layout have been introduced.
}
```

Two details that are easy to get wrong and silently pass every "happy path" test: never assume one **FileChannel.read()/write()** call transfers the entire buffer — both are looped here specifically because a single call can legitimately return fewer bytes than requested, especially under load or on some filesystems. And **readPage** treats **totalRead == 0** (nothing at all was read) as "this page doesn't exist yet," distinct from a short read that still made partial progress.

13. Why fsync Alone Isn't Enough: The WAL-Before-Data-Page Rule

FileChannel.force(true) (**fsync**) guarantees bytes already written to a **FileChannel** are durably on disk — but it says nothing about **ordering** between two different files (a data page's **.tbl** file and the WAL's **.log** file). Calling **force(true)** on a data page file after writing to it is necessary but not sufficient for correctness; the actual durability rule this whole guide is built around is:

WAL record durable FIRST

↓
data page may become durable AFTER (whenever convenient — even much later)

Part 5 builds the WAL manager that makes this ordering concrete; Part 7's recovery algorithm is what makes violating this rule specifically dangerous — a data page containing a change with no corresponding durable WAL record is a change recovery can never verify actually happened.

14. Phase 6 — The Binary Row Format

A row is never stored as a Java object — it's encoded into a flat sequence of bytes before it ever touches a page (§11's **Page.insert(byte[])** already expects exactly this). Using the companion Query Engine guide's **TablePhysicalLayout** (§34 there), every row in a table occupies the **same fixed number of bytes**, laid out at the **same fixed offsets**:

```
+-----+-----+-----+-----+
| null bitmap | id (8B) | name (50B) | age (4B) |
+-----+-----+-----+-----+
```

The null bitmap comes first, sized to the number of columns, and is followed by every column's fixed-width region, in schema order — **TablePhysicalLayout.offset(columnName)** (companion guide §34) gives the exact byte position of each region after the bitmap.

15. Phase 7 — Handling NULLs With a Bitmap

Storing a full extra byte (or worse, a boxed flag) per column just to say "is this null" wastes space badly across millions of rows. A bitmap — one bit per column — is dramatically more compact:

```
columns: id, name, age, email
null bitmap: 00001010
              ^  ^
              |  +--- bit 1 set -> "name" is null
              +----- bit 3 set -> "email" is null
```

```
static void setNullBit(byte[] bitmap, int columnIndex) {
    bitmap[columnIndex / 8] |= (1 << (columnIndex % 8));
}

static boolean isNullBitSet(byte[] bitmap, int columnIndex) {
    return (bitmap[columnIndex / 8] & (1 << (columnIndex % 8))) != 0;
}
```

When a column's null bit is set, its fixed-width byte region is simply never read — its bytes can be anything (typically left as zero) and are ignored entirely, which is why the codec (§16) skips encoding a value at all for a null column rather than writing a sentinel value into its region.

16. Phase 8 — A Real BinaryRowCodec Implementation

```
public interface RowCodec {  
    byte[] encode(Row row);  
    Row decode(byte[] bytes);  
}
```

```
public final class BinaryRowCodec implements RowCodec {  
    private static final int VARCHAR_SLOT_BYTES = 50; // must match TablePhysicalLayout's  
    sizeInBytes(), companion guide §34  
  
    private final TableSchema schema;  
    private final TablePhysicalLayout layout;  
  
    public BinaryRowCodec(TableSchema schema) {  
        this.schema = schema;  
        this.layout = new TablePhysicalLayout(schema);  
    }  
  
    private int nullBitmapBytes() { return (schema.columns().size() + 7) / 8; }  
  
    @Override  
    public byte[] encode(Row row) {  
        byte[] result = new byte[nullBitmapBytes() + layout.slotSize()];  
        ByteBuffer buffer = ByteBuffer.wrap(result);  
  
        for (int i = 0; i < schema.columns().size(); i++) {  
            Column column = schema.columns().get(i);  
            Object value = row.get(column.name());  
            if (value == null) {  
                setNullBit(result, i); // §15 -- and leave this column's byte region untouched (all zero)  
                continue;  
            }  
            int fieldOffset = nullBitmapBytes() + layout.offset(column.name());  
            encodeField(buffer, fieldOffset, column, value);  
        }  
        return result;  
    }  
  
    private void encodeField(ByteBuffer buffer, int offset, Column column, Object value) {  
        switch (column.type()) {  
            case INT -> buffer.putInt(offset, (Integer) value);  
            case BIGINT -> buffer.putLong(offset, (Long) value);  
            case BOOLEAN -> buffer.put(offset, (byte) ((Boolean) value ? 1 : 0));  
            case DECIMAL -> buffer.putDouble(offset, ((Number) value).doubleValue()); // a simplification --  
see §46  
            case DATE, TIMESTAMP -> buffer.putLong(offset, ((Instant) value).toEpochMilli());  
            case VARCHAR -> encodeVarchar(buffer, offset, (String) value);  
        }  
    }  
}
```

```

private void encodeVarchar(ByteBuffer buffer, int offset, String value) {
    byte[] utf8 = value.getBytes(StandardCharsets.UTF_8);
    int maxContentBytes = VARCHAR_SLOT_BYTES - 4; // 4 bytes reserved for a length prefix, §14's diagram
    if (utf8.length > maxContentBytes) {
        throw new IllegalArgumentException("VARCHAR value too long for its fixed slot: \"" + value + "\"");
    }
    buffer.putInt(offset, utf8.length);
    buffer.put(offset + 4, utf8);
}

@Override
public Row decode(byte[] bytes) {
    ByteBuffer buffer = ByteBuffer.wrap(bytes);
    Map<String, Object> values = new LinkedHashMap<>();

    for (int i = 0; i < schema.columns().size(); i++) {
        Column column = schema.columns().get(i);
        if (isNullBitSet(bytes, i)) {
            values.put(column.name(), null);
            continue;
        }
        int fieldOffset = nullBitmapBytes() + layout.offset(column.name());
        values.put(column.name(), decodeField(buffer, fieldOffset, column));
    }
    return new Row(values);
}

private Object decodeField(ByteBuffer buffer, int offset, Column column) {
    return switch (column.type()) {
        case INT -> buffer.getInt(offset);
        case BIGINT -> buffer.getLong(offset);
        case BOOLEAN -> buffer.get(offset) != 0;
        case DECIMAL -> buffer.getDouble(offset);
        case DATE, TIMESTAMP -> Instant.ofEpochMilli(buffer.getLong(offset));
        case VARCHAR -> decodeVarchar(buffer, offset);
    };
}

private String decodeVarchar(ByteBuffer buffer, int offset) {
    int length = buffer.getInt(offset);
    byte[] utf8 = new byte[length];
    buffer.get(offset + 4, utf8);
    return new String(utf8, StandardCharsets.UTF_8);
}
}

```

encode/decode are exact inverses by construction — every field is written and read at the identical **nullBitmapBytes() + layout.offset(columnName)** position, which is precisely why **TablePhysicalLayout** (computed once, at table-creation time, companion guide §34) is passed into this codec rather than recomputed per row: both sides of this round trip must agree on the same offsets, forever, for as long as the table exists.

17. Phase 9 — The Slot Directory, Revisited: Insert and Read, End to End

§11 already built the real slot-directory algorithms inside **Page**. This section is the missing connective piece: how a **Row** (a **Map<String, Object>**) becomes a **RowId**, and back, using **BinaryRowCodec** (§16) and **Page** (§11) together:

Insert:

```
Row --[BinaryRowCodec.encode]--> byte[] --[Page.insert]--> RowId
```

Read:

```
RowId --[Page.read(slotNumber)]--> byte[] --[BinaryRowCodec.decode]--> Row
```

```
public final class PageBasedRowAccess {
    private final RowCodec codec;

    public PageBasedRowAccess(RowCodec codec) { this.codec = codec; }

    public RowId insert(Page page, Row row) {
        byte[] encoded = codec.encode(row);
        return page.insert(encoded); // §11
    }

    public Row read(Page page, int slotNumber) {
        byte[] encoded = page.read(slotNumber); // §11 -- returns null for a tombstoned slot, §32
        return encoded == null ? null : codec.decode(encoded);
    }
}
```

This is the fundamental point-lookup path a **RowId**-based index (companion Query Engine guide §38) ultimately calls into: given a **RowId**, fetch its page (§20's buffer pool decides whether that means a cache hit or a disk read), call **page.read(rowId.slotNumber())**, and decode the result — no scanning, no searching, a direct address-to-value lookup.

18. Reading a Row From Disk, Traced End to End

```
RowId (pageNumber=12, slotNumber=4)
|
v
ThreadPool.fetch(PageId(db, table, 12))          -- §22-23 -- cache hit, or
read from FileChannel via PageIo (§12)
|
v
Page.read(4)                                       -- §11 -- looks up slot 4's
(offset, length), copies those bytes
|
v
BinaryRowCodec.decode(bytes)                      -- §16 -- turns fixed-width
bytes back into a Row
|
v
Row{id: 1, name: "Arpan", age: 32}
```

Every layer built so far — page addressing (§7-8), the slotted layout (§9-11), and the binary codec (§14-16) — appears in this one trace, which is exactly the sequence a **TableScan** (companion Query Engine guide §29) executes once per row while streaming through a table.

19. Phase 10 — DatabaseDirectory and TableFile

One root directory holds everything:

```
tinydb-data/
├── databases/
│   ├── company/
│   │   ├── tables/
│   │   │   ├── employees.tbl
│   │   │   └── departments.tbl
│   │   └── indexes/
│   │       └── employees_pk.idx
│   ├── catalog/
│   │   └── catalog.db
│   ├── wal/
│   │   ├── wal-000001.log
│   │   └── wal-000002.log
│   ├── checkpoints/
│   │   └── checkpoint.meta
│   └── server.lock
```

```
public final class DatabaseDirectory {
    private final Path root;

    public DatabaseDirectory(Path root) { this.root = root; }

    public Path database(String databaseName) {
        return root.resolve("databases").resolve(databaseName);
    }

    public void createDatabase(String databaseName) throws IOException {
        Path db = database(databaseName);
        Files.createDirectories(db.resolve("tables"));
        Files.createDirectories(db.resolve("indexes"));
    }

    public Path tableFile(String databaseName, String tableName) {
        return database(databaseName).resolve("tables").resolve(tableName + ".tbl");
    }
}
```

```
public final class TableFile {
    private final Path path;

    public TableFile(Path path) { this.path = path; }

    public void create() throws IOException {
        Files.createDirectories(path.getParent());
        if (Files.notExists(path)) Files.createFile(path);
    }

    public Path path() { return path; }
}
```

20. Phase 11 — TableFileManager: Owning the FileChannel

```
public final class TableFileManager {
    private final FileChannel channel;

    public TableFileManager(Path path) throws IOException {
        this.channel = FileChannel.open(path, StandardOpenOption.CREATE,
            StandardOpenOption.READ, StandardOpenOption.WRITE);
    }

    public FileChannel channel() { return channel; }

    public long pageCount() throws IOException {
        return channel.size() / StorageConstants.PAGE_SIZE;
    }

    public void close() throws IOException { channel.close(); }
}
```

Everything above this class — the SQL layer, the executors, the planner — must never call **FileChannel** directly. **TableFileManager** is the one place a file handle is opened and owned, exactly the "dependency direction" this guide's §6 architecture diagram already commits to.

21. Phase 12 — FreeSpaceMap: Finding a Page With Room

Before inserting a row, TinyDB needs to find a page with enough free space — scanning every page on every insert would be correct but slow, so a small in-memory map tracks free space per page:

```
public interface FreeSpaceMap {
    Optional<Long> findPage(int requiredBytes);
    void update(long pageNumber, int freeBytes);
}
```

```
public final class SimpleFreeSpaceMap implements FreeSpaceMap {
    private final Map<Long, Integer> freeBytesByPage = new ConcurrentHashMap<>();
    @Override
    public Optional<Long> findPage(int requiredBytes) {
        return freeBytesByPage.entrySet().stream()
            .filter(entry -> entry.getValue() >= requiredBytes)
            .map(Map.Entry::getKey)
            .findFirst(); // a linear scan -- correct, and the honest starting point; see §54 for a bucketed refinement
    }

    @Override
    public void update(long pageNumber, int freeBytes) {
        freeBytesByPage.put(pageNumber, freeBytes);
    }

    public void registerNewPage(long pageNumber) {
        freeBytesByPage.put(pageNumber, StorageConstants.PAGE_SIZE);
    }
}
```

A linear scan over a **Map** is a completely honest "Level 1" implementation — correct, simple, and slow only in the sense that it examines candidates one at a time rather than jumping straight to a good one. §54 names the standard refinement (bucketing pages by free-space percentage) as future work, not a silent gap.

22. Phase 13 — TableHeap: Insert, Read, Update, Delete, Tied Together

TableHeap is the seam between "I have a Row" and "I have a RowId" — every piece built in Parts 1-3 comes together here:

```
public interface TableHeap {
    RowId insert(Row row);
    Row read(RowId rowId);
    void update(RowId rowId, Row row);
    void delete(RowId rowId);
}
```

```
public final class PageBasedTableHeap implements TableHeap {
    private final String database;
    private final String table;
    private final TableFileManager fileManager;
    private final BufferPool bufferPool; // Part 4
    private final FreeSpaceMap freeSpaceMap;
    private final RowCodec codec;
    private final WalManager walManager; // Part 5

    public PageBasedTableHeap(String database, String table, TableFileManager fileManager,
        BufferPool bufferPool, FreeSpaceMap freeSpaceMap, RowCodec codec,
        WalManager walManager) {
        this.database = database;
        this.table = table;
        this.fileManager = fileManager;
        this.bufferPool = bufferPool;
        this.freeSpaceMap = freeSpaceMap;
        this.codec = codec;
        this.walManager = walManager;
    }

    @Override
    public RowId insert(Row row) {
        byte[] encoded = codec.encode(row);
        long pageNumber = findOrCreatePageWithSpace(encoded.length);
        PageId pageld = new PageId(database, table, pageNumber);
        Page page = bufferPool.fetch(pageld);

        // Mutate the in-memory page FIRST so the real, already-assigned slot number is known --
        // the WAL record needs it (redo, §38, must know exactly which slot to replay INSERT into).
        // This does NOT violate §13's rule: that rule governs when the DATA PAGE FILE may become
        // durable on disk relative to the WAL record, not when the in-memory buffer-pool copy may
        // be mutated. LruBufferPool (§25) never flushes this page until later, by which point the
        // WAL append below (and its force(true), if this is a commit) has already happened.
        RowId rowId = page.insert(encoded);
        long lsn = walManager.append(WalRecord.insert(nextTransactionId(), pageld, rowId, encoded));
        page.setPageLsn(lsn);

        freeSpaceMap.update(pageNumber, remainingSpace(page));
        bufferPool.unpin(pageld);
        return rowId;
    }

    @Override
    public Row read(RowId rowId) {
        PageId pageld = new PageId(database, table, rowId.pageNumber());

```

```

    Page page = bufferPool.fetch(pageId);
    try {
        byte[] encoded = page.read(rowId.slotNumber());
        return encoded == null ? null : codec.decode(encoded);
    } finally {
        bufferPool.unpin(pageId);
    }
}

@Override
public void update(RowId rowId, Row row) {
    byte[] afterImage = codec.encode(row);
    PageId pageId = new PageId(database, table, rowId.pageNumber());
    Page page = bufferPool.fetch(pageId);
    byte[] beforeImage = page.read(rowId.slotNumber()); // captured BEFORE mutation -- undo needs
this, §31

    long lsn = walManager.append(WalRecord.update(nextTransactionId(), pageId, rowId,
beforeImage, afterImage));
    page.setPageLsn(lsn);

    page.update(rowId.slotNumber(), afterImage);
    bufferPool.unpin(pageId);
}

@Override
public void delete(RowId rowId) {
    PageId pageId = new PageId(database, table, rowId.pageNumber());
    Page page = bufferPool.fetch(pageId);
    byte[] beforeImage = page.read(rowId.slotNumber()); // the row as it existed right before deletion --
for undo

    long lsn = walManager.append(WalRecord.delete(nextTransactionId(), pageId, rowId,
beforeImage));
    page.setPageLsn(lsn);

    page.delete(rowId.slotNumber());
    bufferPool.unpin(pageId);
}

private long findOrCreatePageWithSpace(int requiredBytes) {
    return freeSpaceMap.findPage(requiredBytes).orElseGet(this::allocateNewPage);
}
// allocateNewPage()/remainingSpace(Page)/nextTransactionId() omitted for brevity --
// allocateNewPage extends the file by one PAGE_SIZE and registers it with the FreeSpaceMap
 (§21).
}

```

update and **delete** follow the pattern **fetch the page, capture the before-image, append the WAL record, set the page's LSN, then mutate the page**; **insert** mutates the in-memory page one step earlier than the other two — specifically so the WAL record can carry the real, already-assigned slot number (needed for redo, §38) rather than the placeholder **-1** it would otherwise have to log before the slot exists. This is not a violation of §13's WAL-before-data-page rule: that rule constrains when the data page's bytes may become durable on disk relative to the WAL record, and **LruBufferPool** (§25) never flushes a page except through its own eviction/**flushAll** logic — by the time that happens, every WAL append (and, for a commit, its **force(true)**) affecting this page has already occurred. The invariant that actually matters, and that all three methods honor without exception, is: **no page is ever flushed to disk before the WAL record describing its most recent change is durable**.

23. Phase 14 — Why a Buffer Pool Exists

Every read in §22's **PageBasedTableHeap** goes through **bufferPool.fetch(pageId)**, never directly through **TableFileManager**'s **FileChannel**. Without a cache in between, reading the same popular page — a table's first page, hit by every query that scans it — would mean a fresh disk read every single time, even though nothing on that page changed since the last read a millisecond ago. A buffer pool caches recently-used pages in memory, and is the single component standing between "one disk read per row accessed" and "one disk read per page, ever, until it's evicted."

24. Phase 15 — BufferFrame and Pin Counts

```
public final class BufferFrame {

    private final Page page;
    private int pinCount;
    private long lastAccessTime;

    public BufferFrame(Page page) {
        this.page = page;
        this.pinCount = 1; // starts pinned -- the caller that just fetched it is using it right now
        this.lastAccessTime = System.nanoTime();
    }

    public Page page() { return page; }

    public void pin() { pinCount++; }
    public void unpin() { pinCount = Math.max(0, pinCount - 1); }
    public boolean isPinned() { return pinCount > 0; }

    public void touch() { lastAccessTime = System.nanoTime(); }
    public long lastAccessTime() { return lastAccessTime; }
}
```

A **pin count**, not a boolean, because the same page can legitimately be in use by more than one concurrent caller at once (a scan reading it, a concurrent write also touching it) — the page is only eligible for eviction once the pin count drops all the way to zero, meaning genuinely nobody is using it right now.

25. Phase 16 — A Real LRU Buffer Pool Implementation

```
public interface BufferPool {
    Page fetch(Pageld pageld);
    void unpin(Pageld pageld);
    void flush(Pageld pageld) throws IOException;
    void flushAll() throws IOException;
}
```

```

public final class LruBufferPool implements BufferPool {
    private final int capacity;
    private final Map<Pageld, BufferFrame> frames; // access-order LinkedHashMap -- eldest-first iteration IS
the LRU order
    private final Map<Pageld, TableFileManager> fileManagers;
    private final Pagelo pagelo = new Pagelo();

    public LruBufferPool(int capacity, Map<Pageld, TableFileManager> fileManagers) {
        this.capacity = capacity;
        this.fileManagers = fileManagers;
        this.frames = new LinkedHashMap<>(capacity, 0.75f, true); // true = access-order, not insertion-order
    }

    @Override
    public synchronized Page fetch(Pageld pageld) {
        BufferFrame frame = frames.get(pageld); // a get() on an access-order LinkedHashMap ALSO
moves it to "most recent"
        if (frame != null) {
            frame.pin();
            frame.touch();
            return frame.page();
        }
        return loadFromDisk(pageld);
    }

    private Page loadFromDisk(Pageld pageld) {
        if (frames.size() >= capacity) evictOnePage();

        try {
            FileChannel channel = fileManagers.get(pageld).channel();
            Page page = pagelo.readPage(channel, pageld);
            frames.put(pageld, new BufferFrame(page));
            return page;
        } catch (IOException e) {
            throw new UncheckedIOException("Failed to load page " + pageld, e);
        }
    }

    private void evictOnePage() {
        // LinkedHashMap with access-order=true iterates ELDEST (least-recently-used) first --
        // exactly the LRU victim-selection order this method needs.
        for (Map.Entry<Pageld, BufferFrame> entry : frames.entrySet()) {
            if (!entry.getValue().isPinned()) {
                evict(entry.getKey(), entry.getValue());
                return;
            }
            // A pinned page is NEVER a valid eviction victim, no matter how stale its access time is --
            // keep scanning forward (toward more-recently-used frames) for the next candidate.
        }
        throw new IllegalStateException("Buffer pool full: every page is pinned, cannot evict");
    }

    private void evict(Pageld pageld, BufferFrame frame) {
        if (frame.page().dirty()) {
            try { flushFrame(pageld, frame); } catch (IOException e) { throw new UncheckedIOException(e);
        }
        frames.remove(pageld);
    }
}

```

```

@Override
public synchronized void unpin(PageId pageId) {
    BufferFrame frame = frames.get(pageId);
    if (frame != null) frame.unpin();
}

@Override
public synchronized void flush(PageId pageId) throws IOException {
    BufferFrame frame = frames.get(pageId);
    if (frame != null && frame.page().dirty()) flushFrame(pageId, frame);
}

@Override
public synchronized void flushAll() throws IOException {
    for (var entry : frames.entrySet()) {
        if (entry.getValue().page().dirty()) flushFrame(entry.getKey(), entry.getValue());
    }
}

private void flushFrame(PageId pageId, BufferFrame frame) throws IOException {
    FileChannel channel = fileManagers.get(pageId).channel();
    pageIo.writePage(channel, frame.page()); // §12
    frame.page().clearDirty();
}
}

```

Using a **LinkedHashMap** constructed with **accessOrder = true** is doing real, load-bearing work here, not just a convenient data structure choice: every **get()** call on such a map automatically re-links that entry to the "most recently used" end of iteration order, which means **evictOnePage**'s simple "iterate from the front" loop is correct LRU victim selection, with zero hand-written linked-list bookkeeping.

26. Phase 17 — The Fetch Algorithm: Cache Hit, Eviction, and the Pinned-Page Rule

```

fetch(PageId)
|
+-- cache hit? (frames.get(pageId) != null)
|   |
|   yes -> pin() -> touch() -> return page
|
+-- no (cache miss)
|   |
|   v
|   is the pool at capacity?
|   |
|   yes -> evictOnePage()
|       |
|       v
|       scan frames, eldest first
|       |
|       v
|       first UNPINNED frame found -> flush if dirty -> remove
|       (a pinned frame is skipped, never evicted, regardless of age)
|
|   v
|   read the page from disk (PageIo.readPage, §12)
|   |
|   v
|   insert a new BufferFrame, pinCount = 1
|   |
|   v
|   return the page

```

27. Phase 18 — Log Sequence Numbers

A **Log Sequence Number (LSN)** is a strictly increasing integer, assigned once per WAL record, in the exact order records are appended. Two facts about LSNs drive every design decision in this Part:

- Every data page records the LSN of the last WAL record that modified it (**Page.setPageLsn**, §11) — this is **pageLSN**.
- Recovery (Part 7) can compare a page's **pageLSN** against the WAL's own durable position to answer, precisely: "does this page already reflect everything the WAL knows about, or does it need a change replayed onto it?"

Nothing here requires LSNs to be globally unique across tables or even across restarts of unrelated concepts — only that within one WAL, they strictly increase, which a single **AtomicLong** counter, incremented once per append, guarantees trivially.

28. Phase 19 — The WAL Record Format

```
public enum WalRecordType {  
    BEGIN, INSERT, UPDATE, DELETE, COMMIT, ABORT, CHECKPOINT  
}
```

```
public record WalRecord(  
    long lsn, // assigned by WalManager.append -- callers pass -1 and never see it until append() returns  
    long transactionId,  
    WalRecordType type,  
    String table, // null for BEGIN/COMMIT/ABORT/CHECKPOINT  
    long pageNumber, // -1 if not applicable  
    int slotNumber, // -1 for BEGIN/COMMIT/ABORT/CHECKPOINT; the real, already-assigned slot for  
INSERT/UPDATE/DELETE  
    byte[] beforeImage, // null unless UPDATE/DELETE -- the row's bytes BEFORE this change, for  
undo (§31)  
    byte[] afterImage // null unless INSERT/UPDATE -- the row's bytes AFTER this change, for redo  
(§30)  
) {  
    public static WalRecord begin(long txId) {  
        return new WalRecord(-1, txId, WalRecordType.BEGIN, null, -1, -1, null, null);  
    }  
    public static WalRecord commit(long txId) {  
        return new WalRecord(-1, txId, WalRecordType.COMMIT, null, -1, -1, null, null);  
    }  
    public static WalRecord abort(long txId) {  
        return new WalRecord(-1, txId, WalRecordType.ABORT, null, -1, -1, null, null);  
    }  
    public static WalRecord insert(long txId, PageId pagId, RowId rowId, byte[] afterImage) {  
        return new WalRecord(-1, txId, WalRecordType.INSERT, pagId.table(), pagId.pageNumber(),  
rowId.slotNumber(), null, afterImage);  
    }  
    public static WalRecord update(long txId, PageId pagId, RowId rowId, byte[] beforeImage, byte[]  
afterImage) {  
        return new WalRecord(-1, txId, WalRecordType.UPDATE, pagId.table(), pagId.pageNumber(),  
rowId.slotNumber(), beforeImage, afterImage);  
    }  
    public static WalRecord delete(long txId, PageId pagId, RowId rowId, byte[] beforeImage) {  
        return new WalRecord(-1, txId, WalRecordType.DELETE, pagId.table(), pagId.pageNumber(),  
rowId.slotNumber(), beforeImage, null);  
    }  
}
```

29. Phase 20 — A Real WalManager Implementation

```
public interface WalManager {
    long append(WalRecord record) throws IOException;
    void flush(long lsn) throws IOException;
    long durableLsn();
    List<WalRecord> readAll() throws IOException; // used by recovery, Part 7
}
```

```
public final class FileWalManager implements WalManager {

    private final FileChannel channel;
    private final AtomicLong nextLsn = new AtomicLong(0);
    private volatile long durableLsn = -1;

    public FileWalManager(Path walFile) throws IOException {
        this.channel = FileChannel.open(walFile, StandardOpenOption.CREATE,
        StandardOpenOption.READ, StandardOpenOption.WRITE);
        this.channel.position(channel.size()); // append at the end of any existing WAL from a prior run
        this.nextLsn.set(recoverNextLsn());
    }

    @Override
    public synchronized long append(WalRecord record) throws IOException {
        // synchronized: WAL appends for ALL transactions must be strictly ordered on disk --
        // exactly the same "one writer at a time" discipline a WAL always requires.
        long lsn = nextLsn.getAndIncrement();
        WalRecord withLsn = new WalRecord(lsn, record.transactionId(), record.type(), record.table(),
            record.pageNumber(), record.slotNumber(), record.beforeImage(), record.afterImage());

        byte[] serialized = serialize(withLsn);
        channel.write(ByteBuffer.wrap(serialized));

        if (record.type() == WalRecordType.COMMIT) {
            channel.force(true); // §30 -- the commit rule: a COMMIT record must be durable before the
            client is told so
            durableLsn = lsn;
        }
        return lsn;
    }

    @Override
    public synchronized void flush(long lsn) throws IOException {
        if (lsn > durableLsn) {
            channel.force(true);
            durableLsn = lsn;
        }
    }

    @Override
    public long durableLsn() { return durableLsn; }

    @Override
    public List<WalRecord> readAll() throws IOException {
        List<WalRecord> records = new ArrayList<>();
        long position = 0;
        while (position < channel.size()) {
```

```

        WalRecord record = readOneRecord(position);
        if (record == null) break; // a torn/truncated tail -- stop here, exactly like §44's checksum
discipline
        records.add(record);
        position += recordLength(record);
    }
    return records;
}

private long recoverNextLsn() throws IOException {
    List<WalRecord> existing = readAll();
    return existing.isEmpty() ? 0 : existing.get(existing.size() - 1).lsn() + 1;
}

private static final int NULL_MARKER = -1;

private byte[] serialize(WalRecord record) {
    byte[] tableBytes = record.table() == null ? null :
record.table().getBytes(StandardCharsets.UTF_8);
    int payloadLength = 8 + 8 + 4 // lsn, transactionId, type ordinal
        + 4 + (tableBytes == null ? 0 : tableBytes.length) // table (length-prefixed)
        + 8 + 4 // pageNumber, slotNumber
        + 4 + (record.beforeImage() == null ? 0 : record.beforeImage().length)
        + 4 + (record.afterImage() == null ? 0 : record.afterImage().length);

    ByteBuffer buffer = ByteBuffer.allocate(4 + payloadLength + 4); // length prefix + payload +
checksum
    buffer.putInt(payloadLength);
    buffer.putLong(record.lsn());
    buffer.putLong(record.transactionId());
    buffer.putInt(record.type().ordinal());
    putLengthPrefixed(buffer, tableBytes);
    buffer.putLong(record.pageNumber());
    buffer.putInt(record.slotNumber());
    putLengthPrefixed(buffer, record.beforeImage());
    putLengthPrefixed(buffer, record.afterImage());

    CRC32 crc = new CRC32();
    crc.update(buffer.array(), 4, payloadLength); // checksum covers the payload only -- not its own
length prefix, not itself
    buffer.putInt((int) crc.getValue());

    return buffer.array();
}

private void putLengthPrefixed(ByteBuffer buffer, byte[] bytes) {
    if (bytes == null) {
        buffer.putInt(NULL_MARKER);
    } else {
        buffer.putInt(bytes.length);
        buffer.put(bytes);
    }
}

private WalRecord readOneRecord(long position) throws IOException {
    ByteBuffer lengthPrefix = ByteBuffer.allocate(4);
    if (readFully(lengthPrefix, position) < 4) return null; // not even a full length prefix left -- torn tail

    int payloadLength = lengthPrefix.getInt(0);
    if (payloadLength <= 0) return null; // a zero/negative length can never be genuine -- treat as

```

corruption, stop

```
    ByteBuffer body = ByteBuffer.allocate(payloadLength + 4); // payload + trailing checksum
    if (readFully(body, position + 4) < body.capacity()) return null; // torn write -- fewer bytes than
    promised
```

```
    int storedChecksum = body.getInt(payloadLength);
    CRC32 crc = new CRC32();
    crc.update(body.array(), 0, payloadLength);
    if ((int) crc.getValue() != storedChecksum) return null; // corrupted record -- never trust it, stop
    scanning here
```

```
    body.limit(payloadLength);
    body.position(0);
    long lsn = body.getLong();
    long transactionId = body.getLong();
    WalRecordType type = WalRecordType.values()[body.getInt()];
    String table = getLengthPrefixedString(body);
    long pageNumber = body.getLong();
    int slotNumber = body.getInt();
    byte[] beforeImage = getLengthPrefixedBytes(body);
    byte[] afterImage = getLengthPrefixedBytes(body);
```

```
    return new WalRecord(lsn, transactionId, type, table, pageNumber, slotNumber, beforeImage,
    afterImage);
}
```

```
private int readFully(ByteBuffer buffer, long position) throws IOException {
    int totalRead = 0;
    while (buffer.hasRemaining()) {
        int read = channel.read(buffer, position + totalRead);
        if (read < 0) break; // end of file -- caller interprets a short read as a torn tail
        totalRead += read;
    }
    return totalRead;
}
```

```
private String getLengthPrefixedString(ByteBuffer buffer) {
    byte[] bytes = getLengthPrefixedBytes(buffer);
    return bytes == null ? null : new String(bytes, StandardCharsets.UTF_8);
}
```

```
private byte[] getLengthPrefixedBytes(ByteBuffer buffer) {
    int length = buffer.getInt();
    if (length == NULL_MARKER) return null;
    byte[] bytes = new byte[length];
    buffer.get(bytes);
    return bytes;
}
```

```
private int recordLength(WalRecord record) {
    // Re-derives the exact on-disk size by re-running the same encoding serialize() already used to
    // write this record -- one source of truth for the byte layout, rather than a second, hand-kept
    // size formula that could silently drift out of sync with serialize() as the format evolves.
    return serialize(record).length;
}
}
```

30. Phase 21 — Appending, Flushing, and the Commit Rule

```
transaction
|
create WAL record (BEGIN, or INSERT/UPDATE/DELETE, or COMMIT)
|
serialize
|
append to WAL file (in memory / OS page cache at this point)
|
assign LSN
|
[for a COMMIT record only] force(true) -- NOW durable on physical
disk
|
return LSN
```

The commit rule: a transaction's **COMMIT** record must be durable — physically on disk, not just written to the OS's page cache — **before** the client is ever told the transaction succeeded. §29's **append** enforces this by calling **channel.force(true)** specifically (and only) when the record being appended is a **COMMIT** — every other record type is written but not immediately forced, which is a deliberate performance choice: forcing on every single **INSERT/UPDATE** would make every write pay **fsync**'s latency cost individually, when only the commit actually needs that guarantee before the client can be told "success."

31. The WAL-Before-Data-Page Rule, Enforced in Code

§13 stated the rule in prose; §22's **PageBasedTableHeap** enforces it structurally, though not by textual line order alone — the remaining piece is why this specific ordering, stated precisely: if page 10 is flushed to disk with **pageLSN = 100**, then before that flush is allowed to happen, the WAL record with **lsn = 100** must **already** be durable. If it isn't — if a crash happens after the data page hits disk but before the WAL record does — recovery (Part 7) has no way to know that change was ever supposed to be durable in the first place, and no way to safely redo or undo it. **LruBufferPool** (§25) never independently decides when to flush a dirty page specifically to keep this invariant simple to reason about: as long as every WAL append that could affect a page happens (and, for commits, is forced) before that page is ever handed to **Pagelo.writePage** (§12), the ordering holds by construction — regardless of whether the in-memory mutation happened microseconds before or after the WAL append, which is a detail the disk-durability rule was never actually about.

32. Phase 22 — INSERT Through the Full Stack

```
INSERT INTO employees VALUES (1, 'Arpan', 'arpan@example.com', 32);
|
v
InsertExecutor (companion Query Engine guide, §17)
|
v
BinaryRowCodec.encode(row) ($16)
|
v
FreeSpaceMap.findPage(requiredBytes) ($21)
|
v
BufferPool.fetch(pageId) ($25) -- cache hit, or PageIo.readPage from disk ($12)
|
v
page.insert(encodedRowBytes) -> RowId ($11) -- the slot directory algorithm from Part 1;
|                                     mutates only the IN-MEMORY page, not yet flushed to disk
v
WalManager.append(WalRecord.insert(..., rowId, ...)) ($29) -- now carries the real slot number; URABLE
|                                     (for commit) before the page is ever
FLUSHED, §31
v
page.setPageLsn(lsn) ($11)
|
v
FreeSpaceMap.update(pageNumber, remainingSpace) ($21)
|
v
BufferPool.unpin(pageId) ($25)
|
v
[later, whenever convenient] page flushed to disk ($12, $25) -- its own durability was never
what the client waited on
```

33. Phase 23 — UPDATE: Before/After Images, Revisited

```
read old row bytes (beforeImage)
|
encode new row bytes (afterImage)
|
WAL UPDATE record carrying BOTH images
|
modify the page in place (fixed-size slots, §11, mean this never moves the
row)
|
mark dirty
```

34. Phase 24 — DELETE: Why Tombstones, Not Physical Erasure

Page.delete (§11) sets a slot's length to a **TOMBSTONE_LENGTH** sentinel rather than physically erasing the row's bytes or compacting the page immediately. Two reasons, both concrete:

- The **RowId must remain a valid, stable reference** for as long as anything else (an index, a WAL undo record, a concurrent scan already positioned at this slot) might still reference it — physically removing the slot entry would shift every subsequent slot's number, silently invalidating every RowId that pointed past it.
- **Undo needs somewhere to restore to**. If a transaction that deleted a row is later rolled back (§39), undo needs to un-delete it — trivial with a tombstone flag (**page.restoreSlot(slotNumber, beforeImage)**), §11, writing the original bytes back and restoring the slot's original length), considerably harder if the bytes were already physically gone.

Reclaiming a tombstoned slot's space is deferred to a background **compaction** pass — the same idea, and the same honest tradeoff, TinyDB's own segment compaction already documents: an append/tombstone-only design defers space reclamation to a later, batched pass rather than paying a compaction cost on every single delete.

35. Why Tombstones Matter Once MVCC Exists

The tombstone discipline in §34 becomes even more important once MVCC (§33's forward reference) is layered on top: a "deleted" row under MVCC isn't gone at all from an older transaction's point of view — a transaction whose snapshot began before the delete must still be able to read the pre-delete version. A physically-erased row makes that structurally impossible; a tombstoned one (or, more generally, a row marked "deleted by transaction X" rather than truly removed) preserves exactly the information an MVCC visibility check needs: when did this row stop existing, and does that boundary fall before or after the reading transaction's own snapshot.

Part 7: Crash Recovery

36. What Crash Recovery Must Guarantee

Every piece built so far — pages, the buffer pool, the WAL — exists to make one promise possible: **after a crash at any arbitrary instant, restarting TinyDB must bring the database back to a state that reflects exactly the committed transactions, and none of the uncommitted ones.** Stated as two separate, equally load-bearing guarantees:

- **Durability:** if a transaction's **COMMIT** record made it to durable storage (§30's commit rule) before the crash, every change that transaction made must be visible after restart — even if the data pages themselves never made it to disk before the crash, because the buffer pool was still holding them dirty in memory.
- **Atomicity:** if a transaction never got a durable **COMMIT** record — because it crashed mid-way, or was explicitly aborted — none of its changes may be visible after restart, even the ones that did make it to a data page on disk before the crash (**LruBufferPool's** eviction, §25, can flush a dirty page from an uncommitted transaction at any time; a buffer pool has no idea which transaction a page's changes belong to, nor should it).

A crash can land at any of these instants relative to what's durable:

committed, page flushed	-> must remain visible	(nothing to do)
committed, page NOT flushed	-> must become visible on restart	(REDO)
uncommitted, page flushed	-> must become invisible again	(UNDO)
uncommitted, page NOT flushed	-> must remain invisible	(nothing to do)

Two of these four cases require recovery to actively do something to the data pages on restart — replay a change that never made it to disk (**redo**), or reverse one that did (**undo**) — and the WAL, built in Part 5 specifically to have both a **beforeImage** and an **afterImage** per record (§28), is the only source of truth recovery has for either direction. This is the exact three-phase shape (Analysis, Redo, Undo) the ARIES recovery algorithm is built around, and the shape this section implements.

37. Phase 25 — Analysis:

Scanning the WAL and Classifying Transactions

Recovery's first pass reads every record the WAL has, in forward order, and answers one question per transaction: **did it commit?**

```
public final class AnalysisResult {
    private final Set<Long> committedTransactionIds = new HashSet<>();
    private final Set<Long> allTransactionIds = new HashSet<>();

    void observe(WalRecord record) {
        allTransactionIds.add(record.transactionId());
        if (record.type() == WalRecordType.COMMIT) {
            committedTransactionIds.add(record.transactionId());
        }
    }

    public Set<Long> committedTransactionIds() { return committedTransactionIds; }

    public Set<Long> incompleteTransactionIds() {
        Set<Long> incomplete = new HashSet<>(allTransactionIds);
        incomplete.removeAll(committedTransactionIds);
        return incomplete;
    }
}
```

38. Phase 26 — Redo:

Replaying Committed History, Idempotently

Redo's job, stated precisely: **make every data page reflect everything the WAL knows happened to it — regardless of whether that transaction ultimately committed or not.** This is the "repeat history" principle ARIES is built on, and it is deliberately not "only redo committed transactions": Undo (§39) is what removes an incomplete transaction's effects, and it can only do that correctly by starting from a page state where everything logged has first been applied, uncommitted changes included — a page half-updated from an interrupted redo would leave nothing coherent for undo to reverse.

Redo walks the WAL **forward**, and for each **INSERT/UPDATE/DELETE** record, applies it to the page — but only if the page doesn't already reflect it:

```
private void redo(WalRecord record, BufferPool bufferPool, String database) throws IOException {
    if (record.type() != WalRecordType.INSERT
        && record.type() != WalRecordType.UPDATE
        && record.type() != WalRecordType.DELETE) {
        return; // BEGIN/COMMIT/ABORT/CHECKPOINT carry no page mutation to replay
    }

    PageId pageId = new PageId(database, record.table(), record.pageNumber());
    Page page = bufferPool.fetch(pageId);
    try {
        if (page.pageLsn() >= record.lsn()) {
            return; // idempotency check -- this page already reflects this change, or a later one; skip it
        }
        switch (record.type()) {
            case INSERT -> page.insert(record.afterImage()); // note: slot allocation, see the caveat below
            case UPDATE -> page.update(record.slotNumber(), record.afterImage());
            case DELETE -> page.delete(record.slotNumber());
            default -> throw new IllegalStateException("unreachable");
        }
        page.setPageLsn(record.lsn());
    } finally {
        bufferPool.unpin(pageId);
    }
}
```

The idempotency check, **page.pageLsn() >= record.lsn()**, is what makes redo safe to run even over records the page already has — which matters because recovery has no way to know in advance exactly which of the changes it's about to replay were already durable on disk before the crash. This is precisely why §11's **initializeNewPage()** initializes a fresh page's pageLSN to the sentinel **-1** rather than **0**: WAL LSNs themselves start at 0 (§29's **AtomicLong nextLsn = new AtomicLong(0)**), so a brand-new page's very first logged change — **lsn = 0** — must compare as not yet applied (**-1 >= 0** is **false**, correctly triggering redo); had the sentinel been **0** instead, **0 >= 0** would evaluate **true** and redo would silently skip the first change ever made to that page.

39. Phase 27 — Undo: Rolling Back Incomplete Transactions

Once redo has brought every page fully up to date with everything the WAL logged, undo removes exactly the changes made by transactions in **incompleteTransactionIds()** (§37) — walking the WAL **backward**, so that a transaction's own changes are undone in the reverse of the order they were made (an **UPDATE** followed by a second **UPDATE** to the same row must have the second one undone first, restoring the first update's after-image, before that in turn is undone back to the row's true original value):

```
private void undo(WalRecord record, BufferPool bufferPool, String database) throws IOException {
    if (record.type() != WalRecordType.INSERT
        && record.type() != WalRecordType.UPDATE
        && record.type() != WalRecordType.DELETE) {
        return;
    }

    PageId pageId = new PageId(database, record.table(), record.pageNumber());
    Page page = bufferPool.fetch(pageId);
    try {
        switch (record.type()) {
            case INSERT -> page.delete(record.slotNumber()); // undo an insert: tombstone it
            case UPDATE -> page.update(record.slotNumber(), record.beforeImage()); // undo an update: restore the
old bytes
            case DELETE -> page.restoreSlot(record.slotNumber(), record.beforeImage()); // undo a delete:
un-tombstone it
            default -> throw new IllegalStateException("unreachable");
        }
        page.setPageLsn(record.lsn());
    } finally {
        bufferPool.unpin(pageId);
    }
}
```

Each case is exactly the inverse operation of what the record originally logged, which is the entire reason **WalRecord** (§28) carries a **beforeImage** at all — without it, undoing an **UPDATE** or a **DELETE** would have nothing to restore to. **INSERT**'s undo needs no before-image because "undo an insert" simply means "this row should never have existed," and **page.delete** (§11's tombstone, not physical erasure — §34) is exactly that.

40. Phase 28 — A Real RecoveryManager, Assembling All Three Phases

```
public final class RecoveryManager {

    private final WalManager walManager;
    private final BufferPool bufferPool;
    private final String database;

    public RecoveryManager(WalManager walManager, BufferPool bufferPool, String database) {
        this.walManager = walManager;
        this.bufferPool = bufferPool;
        this.database = database;
    }

    public void recover() throws IOException {
        List<WalRecord> log = walManager.readAll(); // §29 -- stops cleanly at a torn/truncated tail

        // Phase 1: Analysis
        AnalysisResult analysis = new AnalysisResult();
        for (WalRecord record : log) {
            analysis.observe(record);
        }

        // Phase 2: Redo -- forward order, repeat history for EVERY transaction, committed or not
        for (WalRecord record : log) {
            redo(record, bufferPool, database);
        }

        // Phase 3: Undo -- reverse order, only transactions with no COMMIT record
        Set<Long> incomplete = analysis.incompleteTransactionIds();
        for (int i = log.size() - 1; i >= 0; i--) {
            WalRecord record = log.get(i);
            if (incomplete.contains(record.transactionId())) {
                undo(record, bufferPool, database);
            }
        }

        bufferPool.flushAll(); // §25 -- persist recovery's own corrections before accepting new work
    }

    // redo(...) and undo(...) are §38 and §39 exactly as written there.
}
```

recover() runs once, synchronously, before TinyDB accepts a single new connection — a database that started serving queries against pages recovery hasn't finished fixing up yet would be handing out answers from a state that never actually existed, committed or otherwise. The final **bufferPool.flushAll()** matters for a subtle reason: recovery's own redo/undo mutations are themselves only in-memory until flushed — if TinyDB crashed a second time immediately after recovery finished but before this flush, the next recovery would need to redo recovery's own work all over again, which is correct (idempotency, §38, makes this safe) but wasteful; flushing once at the end of **recover()** avoids that.

41. Testing Recovery: The Crash Test That Must Pass

@Test

```
void committedTransactionSurvivesACrash() throws IOException {
```

```
    // 1. Start TinyDB, insert a row, commit.
```

```
    TinyDbInstance db = TinyDbInstance.start(dataDir);
```

```
    db.execute("INSERT INTO employees VALUES (1, 'Arpan', 'arpan@example.com', 32)");
```

```
    // (the INSERT's WAL record and its transaction's COMMIT record are now durable, §30)
```

```
    // 2. Simulate a crash: stop WITHOUT a clean shutdown -- no final buffer pool flush.
```

```
    db.crashWithoutFlushing();
```

```
    // 3. Restart. RecoveryManager.recover() runs automatically on startup.
```

```
    TinyDbInstance restarted = TinyDbInstance.start(dataDir);
```

```
    // 4. The row must be visible -- REDO must have replayed it, because the crash happened
```

```
    //    before the dirty page holding it was ever flushed to disk.
```

```
    Row row = restarted.execute("SELECT * FROM employees WHERE id = 1").singleRow();
```

```
    assertEquals("Arpan", row.get("name"));
```

```
}
```

@Test

```
void uncommittedTransactionIsInvisibleAfterACrash() throws IOException {
```

```
    // 1. Start TinyDB, insert a row, but crash BEFORE committing.
```

```
    TinyDbInstance db = TinyDbInstance.start(dataDir);
```

```
    db.beginTransaction();
```

```
    db.execute("INSERT INTO employees VALUES (2, 'Priya', 'priya@example.com', 29)");
```

```
    db.crashWithoutFlushing(); // no COMMIT record was ever appended
```

```
    // 2. Restart. RecoveryManager.recover() runs UNDO for this incomplete transaction.
```

```
    TinyDbInstance restarted = TinyDbInstance.start(dataDir);
```

```
    // 3. The row must NOT be visible.
```

```
    QueryResult result = restarted.execute("SELECT * FROM employees WHERE id = 2");
```

```
    assertTrue(result.rows().isEmpty());
```

```
}
```

Part 8: Checkpoints and Corruption Detection

43. Phase 29 — A Real (Quiescent) Checkpoint Implementation

```
public static WalRecord checkpoint() {  
    return new WalRecord(-1, -1, WalRecordType.CHECKPOINT, null, -1, -1, null, null);  
}
```

```
public final class CheckpointManager {  
    private final WalManager walManager;  
    private final BufferPool bufferPool;  
  
    public CheckpointManager(WalManager walManager, BufferPool bufferPool) {  
        this.walManager = walManager;  
        this.bufferPool = bufferPool;  
    }  
  
    /**  
     * Quiescent checkpoint: the CALLER is responsible for ensuring no transaction is  
     * currently active before invoking this (e.g. a background scheduler that briefly  
     * pauses new transaction starts) -- this is a "Level 1" tradeoff (§54, "Future Enhancements"):  
     * production systems use a "fuzzy" checkpoint that doesn't require quiescing at all,  
     * at the cost of a checkpoint record that must also list which transactions were  
     * still active, which recovery then still has to walk further back to find.  
     */  
    public long takeCheckpoint() throws IOException {  
        bufferPool.flushAll(); // every dirty page is now durable -- §31's rule, satisfied for ALL pages at  
        once  
        return walManager.append(WalRecord.checkpoint());  
    }  
}
```

RecoveryManager.recover() (§40) is extended with exactly one step, inserted before Analysis: find the last CHECKPOINT record in the log, if any, and discard everything before it before Analysis, Redo, or Undo ever see it —

```
public void recover() throws IOException {  
    List<WalRecord> fullLog = walManager.readAll();  
    List<WalRecord> log = afterLastCheckpoint(fullLog); // §43 -- everything before it is provably already durable  
  
    AnalysisResult analysis = new AnalysisResult();  
    for (WalRecord record : log) {  
        analysis.observe(record);  
    }  
    for (WalRecord record : log) {  
        redo(record, bufferPool, database);  
    }  
    Set<Long> incomplete = analysis.incompleteTransactionIds();  
    for (int i = log.size() - 1; i >= 0; i--) {  
        WalRecord record = log.get(i);  
        if (incomplete.contains(record.transactionId())) {  
            undo(record, bufferPool, database);  
        }  
    }  
    bufferPool.flushAll();  
}
```

```

private List<WalRecord> afterLastCheckpoint(List<WalRecord> fullLog) {
    int lastCheckpointIndex = -1;
    for (int i = 0; i < fullLog.size(); i++) {
        if (fullLog.get(i).type() == WalRecordType.CHECKPOINT) {
            lastCheckpointIndex = i;
        }
    }
    return lastCheckpointIndex == -1 ? fullLog : fullLog.subList(lastCheckpointIndex + 1, fullLog.size());
}

```

44. Phase 30 — Page Checksums: CRC32 Corruption Detection, For Real

```

public final class PageIo {
    private static final int CHECKSUM_OFFSET = 32;
    private static final int CHECKSUM_LENGTH = 4;

    // ... readPage(...) / writePage(...) exactly as in §12 ...

    private int computeChecksum(Page page) {
        CRC32 crc = new CRC32();
        ByteBuffer buffer = page.buffer().duplicate();

        ByteBuffer beforeChecksumField = buffer.duplicate();
        beforeChecksumField.limit(CHECKSUM_OFFSET).position(0);
        crc.update(beforeChecksumField);

        ByteBuffer afterChecksumField = buffer.duplicate();
        afterChecksumField.position(CHECKSUM_OFFSET +
CHECKSUM_LENGTH).limit(StorageConstants.PAGE_SIZE);
        crc.update(afterChecksumField);

        return (int) crc.getValue();
    }

    void verifyChecksum(Page page) {
        int expected = page.checksum();
        int actual = computeChecksum(page);
        if (expected != actual) {
            throw new CorruptedPageException(page.pageId(), expected, actual);
        }
    }

    void computeAndStoreChecksum(Page page) {
        page.setChecksum(computeChecksum(page));
    }
}

```

```

public final class CorruptedPageException extends RuntimeException {
    public CorruptedPageException(Pageld pageId, int expectedChecksum, int actualChecksum) {
        super("Page " + pageId + " failed checksum verification: expected " + expectedChecksum
            + " but computed " + actualChecksum + " -- refusing to serve possibly-corrupted data");
    }
}

```

Part 9: The Storage Engine Facade

45. Phase 31 — The StorageEngine Interface and a Real Implementation

Every layer built across Parts 1-8 — pages, rows, table heaps, the buffer pool, the WAL, recovery, checkpoints, checksums — is an implementation detail the SQL layer (companion Query Engine guide) should never have to know about directly. StorageEngine is the one seam: the entire, single door through which the rest of TinyDB reaches disk.

```
public interface StorageEngine {
    void createDatabase(String database) throws IOException;
    void createTable(String database, String table) throws IOException;

    TableHeap tableHeap(String database, String table); // §22 -- insert/read/update/delete by RowId

    void beginTransaction(long transactionId) throws IOException;
    void commitTransaction(long transactionId) throws IOException;
    void abortTransaction(long transactionId) throws IOException;

    void recover() throws IOException; // §40/§43 -- run once, at startup, before anything else
    void shutdown() throws IOException; // flush everything, close every file cleanly
}
```

```
public final class FileStorageEngine implements StorageEngine {

    private final Path dataRoot;
    private final DatabaseDirectory directory;
    private final Map<String, TableFileManager> fileManagers = new ConcurrentHashMap<>();
    private final Map<String, WalManager> walManagers = new ConcurrentHashMap<>(); // one
WAL per database
    private final Map<String, TableHeap> tableHeaps = new ConcurrentHashMap<>();
    private final BufferPool bufferPool; // one shared pool, all tables

    public FileStorageEngine(Path dataRoot, int bufferPoolCapacity) {
        this.dataRoot = dataRoot;
        this.directory = new DatabaseDirectory(dataRoot);
        this.bufferPool = new LruBufferPool(bufferPoolCapacity, new PageIo());
    }

    @Override
    public void createDatabase(String database) throws IOException {
        directory.createDatabase(database);
        walManagers.put(database, new
FileWalManager(directory.database(database).resolve("wal").resolve("wal.log")));
    }

    @Override
    public void createTable(String database, String table) throws IOException {
        Path path = directory.tableFile(database, table);
        new TableFile(path).create();

        TableFileManager fileManager = new TableFileManager(path);
        fileManagers.put(key(database, table), fileManager);

        TableHeap heap = new PageBasedTableHeap(database, table, fileManager, bufferPool,
            new SimpleFreeSpaceMap(), new BinaryRowCodec(/* TablePhysicalLayout for this table */)
    }
```

```

null),
    walManagers.get(database));
    tableHeaps.put(key(database, table), heap);
}

@Override
public TableHeap tableHeap(String database, String table) {
    return tableHeaps.get(key(database, table));
}

@Override
public void beginTransaction(long transactionId) throws IOException {
    // one WAL per database in this design -- a cross-database transaction would need a
    // distributed-commit protocol this guide doesn't build; see §54.
    for (WalManager wal : walManagers.values()) wal.append(WalRecord.begin(transactionId));
}

@Override
public void commitTransaction(long transactionId) throws IOException {
    for (WalManager wal : walManagers.values()) wal.append(WalRecord.commit(transactionId));
}

@Override
public void abortTransaction(long transactionId) throws IOException {
    for (WalManager wal : walManagers.values()) wal.append(WalRecord.abort(transactionId));
}

@Override
public void recover() throws IOException {
    for (Map.Entry<String, WalManager> entry : walManagers.entrySet()) {
        new RecoveryManager(entry.getValue(), bufferPool, entry.getKey()).recover(); // §40/§43
    }
}

@Override
public void shutdown() throws IOException {
    bufferPool.flushAll();
    for (TableFileManager fileManager : fileManagers.values()) fileManager.close();
}

private String key(String database, String table) { return database + "." + table; }
}

```